AWS DMS | FULL LOAD + CDC | END TO END

SQL Server Migration with AWS DMS

A DBA guide from assessment to cutover

Architecture, source and target preparation, task build, validation, cutover, rollback, heterogeneous migration and troubleshooting



WHAT YOU WILL LEARN

- DMS architecture, CDC modes and Serverless vs instances
- Assessment, SQL Server limitations and planning
- Source and target preparation (CDC, schema, identity)
- Building tasks, mappings, settings and validation
- Cutover, rollback, clean-up and SQL Server to PostgreSQL

Prepared October 2026. Versions, limits and features change often; verify exact values in the AWS documentation.

Contents

How to Use This Guide . . . 4

1. AWS DMS and SQL Server: The Big Picture . . . 5

1.1 What DMS does well . . . 5

1.2 What DMS does not do . . . 5

1.3 DMS or something else? . . . 6

1.4 The end-to-end workflow . . . 6

2. DMS Architecture and Concepts . . . 7

2.1 Building blocks . . . 7

2.2 Network design . . . 7

2.3 Task types and phases . . . 8

2.4 How CDC works for SQL Server . . . 9

2.5 Replication instance or Serverless? . . . 10

3. Assessment and Planning . . . 11

3.1 Inventory and discovery . . . 11

3.2 Choose target and method . . . 11

3.3 SQL Server source limitations to check . . . 12

3.4 Discovery queries to run on the source . . . 12

3.5 Sizing and timing . . . 13

4. Preparing the Source SQL Server . . . 14

4.1 Source readiness checklist . . . 14

4.2 Create the DMS login . . . 14

4.3 Recovery model and transaction log management . . . 15

4.4 Enable change capture . . . 15

4.5 Source endpoint settings you may need . . . 16

4.6 Common source problems . . . 17

5. Preparing the Target . . . 18

5.1 Target SQL Server (RDS or EC2) . . . 18

5.2 Schema: what to create before, during and after . . . 18

5.3 Identity columns and sequences . . . 19

5.4 Other target types in brief . . . 20

5.5 Target readiness checklist . . . 20

6. Building the DMS Migration, Step by Step . . . 21

6.1 Step 1: IAM roles and prerequisites . . . 21

6.2 Step 2: Subnet group and replication instance . . . 21

6.3 Step 3: Endpoints . . . 21
6.4 Step 4: Table mappings . . . 22
6.5 Step 5: Task settings . . . 22
6.6 Step 6: Premigration assessment . . . 23
6.7 Step 7: Create and start the task . . . 23
6.8 Alternative: DMS Serverless . . . 23
7. Running the Migration: Full Load, CDC and Validation . . . 25
7.1 What to expect . . . 25
7.2 Monitoring . . . 25
7.3 Table and task states . . . 26
7.4 Data validation . . . 26
7.5 Performance tuning . . . 27
7.6 Managing a running task . . . 27
8. Cutover, Rollback and Clean-up . . . 28
8.1 Readiness criteria . . . 28
8.2 Cutover sequence . . . 28
8.3 Objects DMS did not move . . . 29
8.4 Rollback options . . . 29
8.5 Clean-up . . . 29
8.6 After cutover: stabilise . . . 30
9. Heterogeneous Migration: SQL Server to PostgreSQL . . . 31
9.1 Choosing the approach . . . 31
9.2 Schema conversion with DMS Schema Conversion . . . 32
9.3 Typical code conversions . . . 32
9.4 Data type mapping (typical) . . . 33
9.5 Loading data into PostgreSQL . . . 33
9.6 Heterogeneous migration checklist . . . 34
10. Troubleshooting and Best Practices . . . 35
10.1 Troubleshooting guide . . . 35
10.2 Reading task logs . . . 36
10.3 Best practices . . . 36
Appendix A: DMS CLI Cheat Sheet . . . 37
Appendix B: Pre-Cutover Checklist . . . 37
Appendix C: Glossary . . . 37
Appendix D: Where to Learn More . . . 38

How to Use This Guide

This guide takes a SQL Server DBA through an end-to-end migration with AWS Database Migration Service (DMS): choosing DMS, assessing the source, preparing source and target, building and tuning the task, validating, cutting over, rolling back and cleaning up. It also covers migrating SQL Server to PostgreSQL.

Chapter	You will learn	Read it when you...
1. Big Picture	What DMS does, what it does not, DMS vs native tools, the workflow	choose a method
2. Architecture	Instances, Serverless, endpoints, tasks, networking, CDC modes	design the solution
3. Assessment	Discovery queries, SQL Server limitations, sizing	plan the project
4. Source Preparation	Login, recovery model, log management, CDC, endpoint settings	prepare SQL Server
5. Target Preparation	Schema ordering, identity, triggers/FKs, RDS settings	prepare the target
6. Building DMS	Roles, instance, endpoints, mappings, settings, assessment, tasks	build the migration
7. Running It	Monitoring, validation, tuning, managing tasks	operate the task
8. Cutover	Runbook, objects DMS skips, rollback, clean-up, stabilise	go live
9. SQL Server to PostgreSQL	Schema Conversion, code and type mapping, loading	change engines
10. Troubleshooting	Symptoms, logs, best practices	something is wrong

The key point

DMS moves **data**, not the whole database. Plan to create the schema, secondary objects, logins and jobs yourself (or convert them for a different engine), and treat cutover as a rehearsed runbook.

A note on accuracy

DMS evolves quickly: engine versions (3.5.x, 3.6.x), supported editions, limitations and features change. The AWS DMS Fleet Advisor reached end of support on 20 May 2026. Values in this guide are planning guidance; confirm limitations, settings and command syntax in the current AWS documentation before building.

1. AWS DMS and SQL Server: The Big Picture

AWS Database Migration Service (DMS) moves data between databases with very little downtime. It reads the source, bulk-copies the existing rows (**full load**), then keeps the target in sync by streaming ongoing changes (**change data capture, CDC**) until you are ready to switch. This guide shows a DBA how to plan, build, run and finish a SQL Server migration with DMS from start to end.

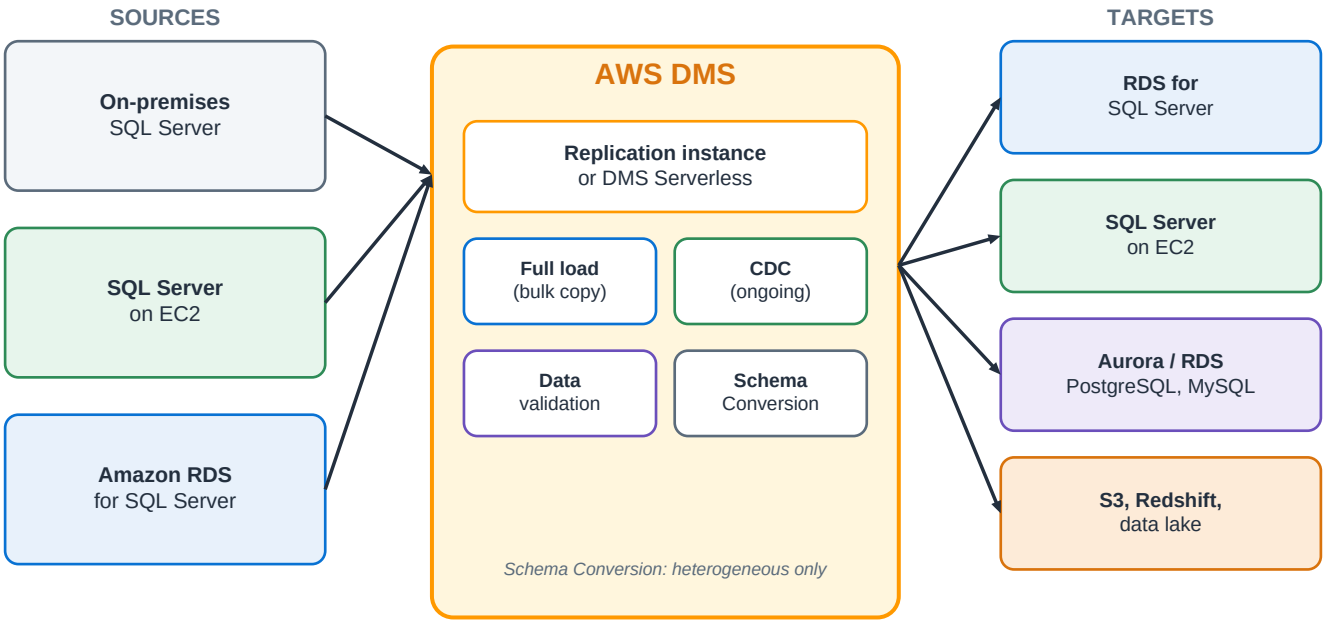


Figure 1.1 - AWS DMS between SQL Server sources and AWS targets

1.1 What DMS does well

- **Low-downtime migrations** - the source stays online during the copy; downtime is the cutover only.
- **Homogeneous and heterogeneous** - SQL Server to SQL Server (RDS or EC2), and SQL Server to Aurora/RDS PostgreSQL, MySQL, S3, Redshift and more.
- **Selective migration** - choose schemas, tables, filters and column transformations with table mappings.
- **Continuous replication** - keep a target in sync for reporting, consolidation or a data lake, not only for migration.
- **Built-in validation** - compare source and target rows during the migration.
- **Managed or serverless** - run on a replication instance you size, or on DMS Serverless that scales for you.

1.2 What DMS does not do

DMS is a **data movement** service. It is not a full database clone. Knowing the gaps early avoids last-minute surprises:

Not moved by DMS (for SQL Server)	How you handle it
Secondary indexes, foreign keys, defaults, check constraints, triggers (the target tables are created in a basic form)	Create the full schema before the load, or generate it from scripts / DMS Schema Conversion
Identity property, computed-column behaviour, sequences, user-defined types	Pre-create tables with identity; re-seed after cutover

Not moved by DMS (for SQL Server)	How you handle it
Stored procedures, functions, views, jobs, logins, linked servers, Database Mail, SSIS	Script and re-create; for heterogeneous targets, convert code
Temporal tables, sparse columns, partition switching, column-level encryption	Not supported as a source; handle separately (section 3.3)
TRUNCATE TABLE operations during CDC	Not captured; plan application behaviour or reload the table
Data masking (masked values migrate unmasked)	Re-apply masking on the target

1.3 DMS or something else?

For SQL Server there are several migration tools. DMS is the right tool when you need **ongoing replication, selective tables or a different target engine**. For a straightforward same-engine move, native backup/restore is often simpler and faster.

Scenario	Best fit	Why
SQL Server to RDS for SQL Server, same version, whole database, short outage OK	Native backup/restore	Fastest, exact copy of schema and data
Same, but minutes of downtime	Native full + diff + log restore, or **DMS**	Both give low downtime; DMS needs more setup but is logical and selective
Only some tables or schemas	**DMS**	Table mappings and filters
Upgrade/consolidate many databases with ongoing sync	**DMS**	Continuous replication, per-table control
SQL Server to Aurora or RDS PostgreSQL / MySQL	**DMS** + DMS Schema Conversion	Heterogeneous; cross-engine data type mapping
Replicate SQL Server to S3 or Redshift (data lake / analytics)	**DMS**	CDC to analytic targets
Move the whole server unchanged to EC2	AWS MGN	Server-level rehost
Express edition source	Native tools or BACPAC	DMS does not support SQL Server Express as a source

1.4 The end-to-end workflow

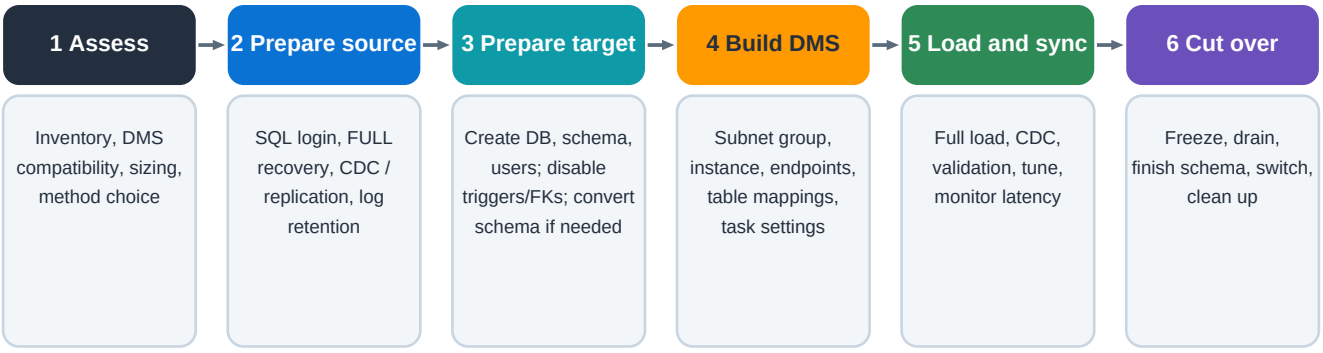


Figure 1.2 - Six stages of a DMS migration for SQL Server

2. DMS Architecture and Concepts

A DMS migration has four building blocks: **compute** to run the work, **endpoints** that describe the source and target, a **task** that defines what to move, and **networking and security** that connect them.

2.1 Building blocks

Component	What it is	DBA notes
Replication instance	An EC2 instance managed by DMS that runs your tasks	You pick the class, storage, VPC, Multi-AZ. Memory is critical for CDC and LOBs.
DMS Serverless	DMS provisions and scales capacity automatically in **DMS Capacity Units (DCUs)**; one DCU is about 2 GB RAM	You set min and max DCUs. Good for variable workloads and when you do not want to size an instance.
Replication subnet group	The subnets (in 2+ AZs) where DMS can place the instance	Use private subnets with routes to source and target.
Endpoint	Connection details for a source or target database	Includes credentials, SSL mode and engine-specific settings. Use Secrets Manager.
Replication task	Definition of what to migrate: type, table mappings and settings	Types: full load, CDC only, or full load + CDC.
Table mappings	JSON rules selecting tables and transforming names/columns	Selection, transformation, filter and parallel-load rules.
Task settings	JSON controlling behaviour: LOBs, load mode, validation, logging, error handling	The main performance and reliability knobs.
Control tables	awsdms_* tables DMS creates on the target	Status, exceptions, suspended tables, history. Useful for troubleshooting.
Premigration assessment	Checks a task for known problems before you run it	Run it before the first start.

2.2 Network design

The replication instance (or serverless capacity) lives in your VPC. It needs network paths to **both** the source and the target. For an on-premises source the path is Site-to-Site VPN or Direct Connect; the on-premises firewall must allow the DMS private IP address to reach SQL Server (port 1433 or your instance's port).

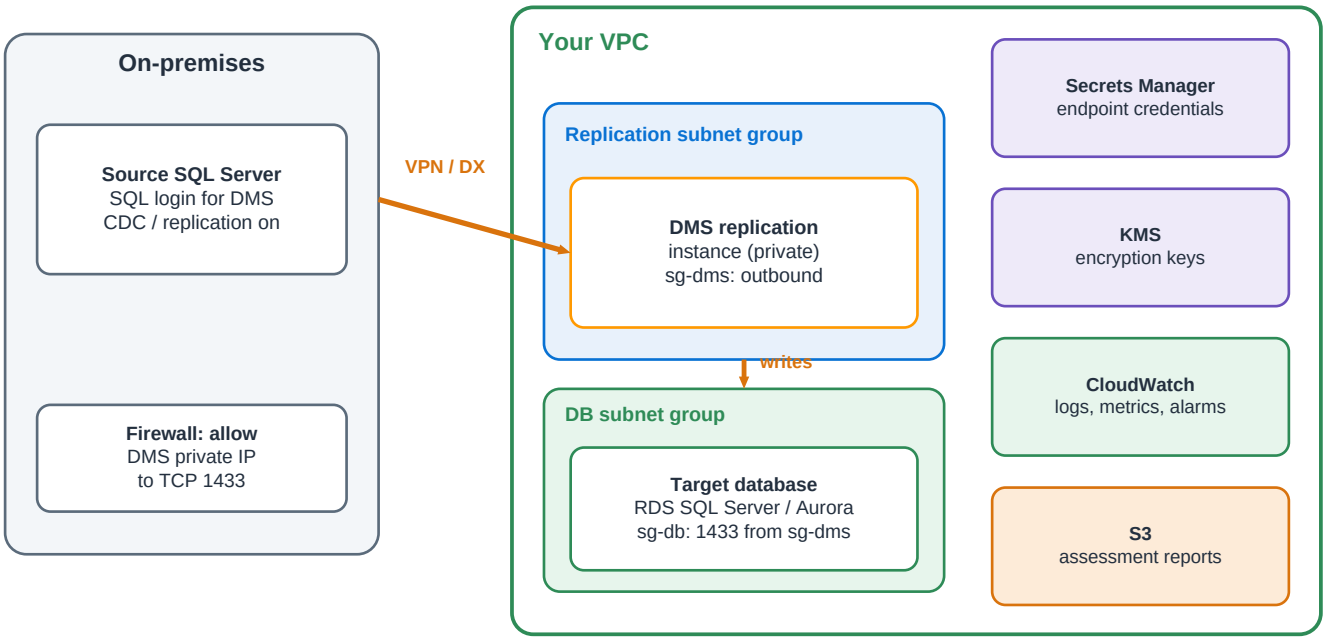


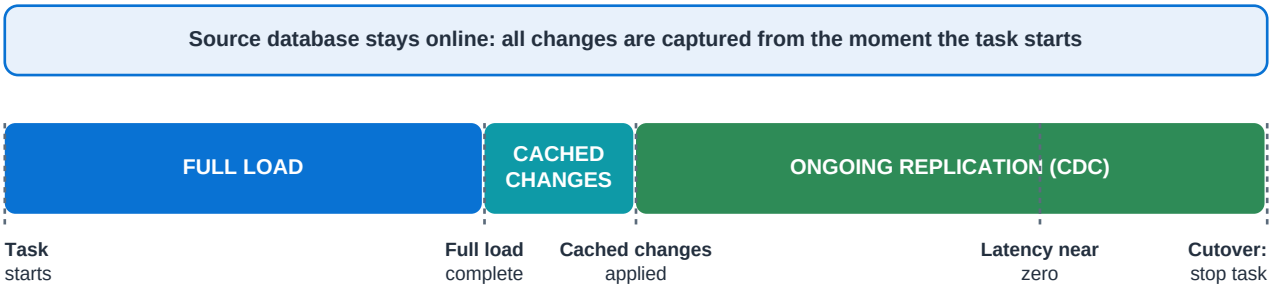
Figure 2.1 - Typical network design for on-premises to AWS

- Keep the replication instance **private**. Do not make it publicly accessible for production migrations.
- Use **security groups**: allow outbound from the DMS group; allow inbound 1433 on the target from the DMS group only.
- Choose **SSL** on endpoints (require or verify-full). For SQL Server sources, import the server certificate if you verify it.
- Keep the instance in the **same Region and ideally the same AZ** as the target to reduce write latency. Large initial loads are often faster when the instance is close to the target.
- Open ports for **named instances**: use the actual port, and the 'IP\InstanceName' notation where needed.

2.3 Task types and phases

Every task is one of three **migration types**. Most SQL Server migrations use **full load + CDC**.

Migration type	What happens	Use it for
Full load	Copies existing data once and stops	Small databases, dev/test, one-time loads
CDC only	Starts reading changes from a given point	When the data was already copied by another method (for example a native restore)
Full load + CDC	Copies data, then applies changes made during the copy and continues	Minimal-downtime production migrations



Users keep working throughout; downtime is limited to the final cutover window.

Figure 2.2 - Phases of a full load + CDC task

During the full load, DMS also **caches** changes happening on the source. When the load completes, it applies the cached changes and then continues in CDC mode. Tables load in parallel (by default up to eight at a time), and each table can fail or finish independently.

2.4 How CDC works for SQL Server

SQL Server does not give outside tools direct access to its log, so DMS uses a feature of SQL Server itself to expose changes. There are two modes:

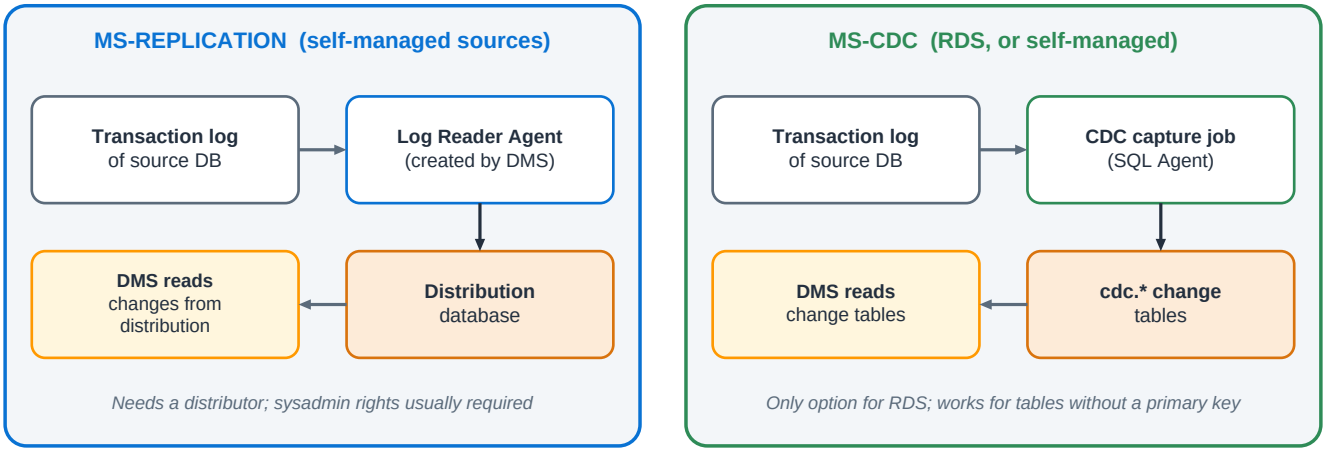


Figure 2.3 - The two ways DMS captures changes from SQL Server

	MS-REPLICATION	MS-CDC
Where	Self-managed sources (on-premises, EC2)	Amazon RDS for SQL Server (the only option), and self-managed
Mechanism	DMS creates a publication and uses a distribution database and the Log Reader Agent	SQL Server's change data capture writes changes to cdc.* tables; DMS reads them
Tables without a primary key	Only inserts and deletes are captured	Updates are captured too
Permissions	Typically sysadmin on the source	Can be set up with lower privileges (see chapter 4)
Side effect	Replication artifacts remain on the source if you delete the task: clean up	CDC jobs and tables consume space and log; clean up at the end
Log retention	Log cannot be truncated before the reader has consumed it	Same; plus CDC cleanup job retention

2.5 Replication instance or Serverless?

	Replication instance	DMS Serverless
Sizing	You choose class and storage	DMS provisions DCUs between your min and max
Billing	Per hour for the instance and storage	Per DCU-hour actually used
Best for	Predictable steady loads, tight control	Variable CDC volume, many tasks, less capacity planning
Control	Full control of class, Multi-AZ, maintenance	Fewer knobs; set min/max capacity, VPC, subnet group
Typical choice for SQL Server	Large one-time migrations with known sizing	Heterogeneous and long-running replication with fluctuating load

Choosing a replication instance class

- **dms.t3 (burstable)**: test and small migrations only; watch CPU credits.
- **dms.c (compute-optimised)**: many small rows, CPU-bound transformation work.
- **dms.r (memory-optimised)**: recommended for production CDC, large transactions and LOB-heavy tables because changes are held in memory before being applied.
- **Storage:** logs and swapped changes use instance storage; allocate more than the default for long-running or busy tasks.
- **Multi-AZ** protects the replication instance itself for long-running replication; it is not needed for a short one-time load.
- Use a **current DMS engine version** (the latest 3.6.x release adds SQL Server improvements); choose automatic minor upgrades with care during a migration.

3. Assessment and Planning

Most DMS problems are found in the first week of testing, not on cutover day, if you assess the source and agree the method before building anything.

3.1 Inventory and discovery

Question	Why it matters	How to find out
SQL Server version and edition	DMS supports sources from SQL Server 2008 to 2022. CDC needs Enterprise, Developer or Standard (2016 and later). Web supports full load only. **Express is not supported**	SERVERPROPERTY
Databases, sizes, growth, busiest tables	Instance sizing, load time, table partitioning strategy	sys.master_files, sys.dm_db_partition_stats
Tables without primary keys	Affect CDC accuracy, validation and batch apply	Catalog query (section 3.4)
LOB columns (varchar(max), xml, varbinary(max), geography)	Determine LOB mode and memory needs	sys.columns
Change rate and peak writes	CDC latency and instance class	Perfmon, log generation, backup sizes
Features DMS does not support	Temporal tables, sparse columns, ADR, column-level encryption	Catalog queries below
Dependencies and cutover constraints	Applications, jobs, ETL, linked servers	Application owners
Target choice and version	RDS SQL Server version >= source, or PostgreSQL / S3 etc.	Requirements

*The older **DMS Fleet Advisor** discovery capability reached end of support on 20 May 2026. For new fleet-level assessments AWS recommends **AWS Migration Evaluator**. For individual databases the SQL checks in this chapter are enough.*

3.2 Choose target and method

Target	Schema approach	Notes
Amazon RDS for SQL Server	Create schema from scripts or restore a schema-only copy; DMS loads data	Most common homogeneous target. Use native backup/restore if you need an exact copy with no CDC.
SQL Server on EC2	Same	Full control of the target; you manage the server.
Aurora / RDS for PostgreSQL (or MySQL)	DMS Schema Conversion (free) or AWS SCT converts schema and code	Heterogeneous; plan application changes. See chapter 9.
Amazon S3 / Redshift	Target format and tables defined by DMS settings	For data lakes and analytics; CDC supported.

3.3 SQL Server source limitations to check

These limitations come from the DMS documentation for SQL Server sources at the time of writing. Check the current page for your DMS version.

Limitation	Impact and action
Windows Authentication is not supported for the source endpoint	Create a SQL login for DMS
Identity property is not migrated	Pre-create target tables with identity; reseed after load
Sparse columns, temporal tables, partition switching, column-level encryption are not supported	Redesign, export separately, or use another method for those objects
Changes to computed columns are not replicated	Make sure the target computes the same values
WRITETEXT / UPDATETEXT changes are not captured	Avoid these legacy statements during the migration
TRUNCATE TABLE is not captured	Stop truncations during CDC or reload the table
Accelerated Database Recovery (ADR) databases are not supported for replication	Disable ADR on the source for the migration window, or use another method
Geometry and geography columns are not supported in full LOB mode	Use limited LOB mode or inline LOB mode
Tables or schemas whose names contain certain special characters	Rename or exclude
Without MS-REPLICATION or MS-CDC, tables without a PK capture only inserts and deletes	Add primary keys or use MS-CDC
Log events can be lost if the log is truncated or archived before DMS reads them	Control log backup/truncation (chapter 4)
CDC on RDS Proxy for SQL Server is not supported	Connect DMS directly to the RDS endpoint
Masked data migrates unmasked	Re-apply masking on the target

3.4 Discovery queries to run on the source

```
-- Edition and version (Express is not supported; Web = full load only)
SELECT SERVERPROPERTY('Edition') AS edition, SERVERPROPERTY('ProductVersion') AS version;

-- Tables without a primary key (affect CDC, validation and batch apply)
SELECT s.name AS schema_name, t.name AS table_name
FROM sys.tables t JOIN sys.schemas s ON s.schema_id = t.schema_id
WHERE NOT EXISTS (SELECT 1 FROM sys.indexes i WHERE i.object_id = t.object_id AND i.is_primary_key = 1);

-- Features DMS cannot handle: temporal tables, sparse columns, ADR
SELECT name, temporal_type_desc FROM sys.tables WHERE temporal_type <> 0;
SELECT OBJECT_NAME(object_id) AS tbl, name AS col FROM sys.columns WHERE is_sparse = 1;
SELECT name, is_accelerated_database_recovery_on FROM sys.databases;   -- SQL Server 2019+

-- LOB columns and their max sizes (-1 = MAX)
SELECT OBJECT_NAME(c.object_id) AS tbl, c.name AS col, ty.name AS type, c.max_length
FROM sys.columns c JOIN sys.types ty ON ty.user_type_id = c.user_type_id
WHERE (ty.name IN ('xml','geography','geometry','text','ntext','image')) OR (c.max_length = -1);

-- Biggest tables (plan parallel load)
SELECT TOP 20 OBJECT_NAME(object_id) AS tbl, SUM(row_count) AS rows
FROM sys.dm_db_partition_stats WHERE index_id IN (0,1) GROUP BY object_id ORDER BY rows DESC;
```

3.5 Sizing and timing

- **Load time** is driven by the slowest of: source read speed, network, replication instance, and target write speed. Test with a representative table to measure rows/second, then extrapolate.
- **Network:** estimate the data size against your VPN/Direct Connect throughput (see the bandwidth table in the MGN guide: 1 TB needs about 2.5 hours at 1 Gbps ideal). Plan for 60-80% efficiency.
- **CDC rate:** know peak log generation. If changes arrive faster than DMS can apply them, latency grows. Size the instance (memory) and consider batch apply.
- **Cutover window:** latency must be close to zero before you stop writes. Plan the window around how long the final drain and validation take.
- **Rehearse:** run the full process once end to end on a copy, record timings, and tune task settings before production.

4. Preparing the Source SQL Server

DMS reads your source while it keeps running. Preparing the source means giving DMS a safe login, making sure the transaction log is kept long enough, and switching on change capture.

4.1 Source readiness checklist

Done?	Item	Why
[ ]	Edition supports CDC (Enterprise, Developer, Standard 2016+). Not Express	Required for ongoing replication
[ ]	Database in FULL recovery model	CDC/replication read the log
[ ]	Dedicated SQL login for DMS (SQL authentication) with the right permissions	Windows Authentication is not supported
[ ]	CDC or MS-REPLICATION configured (section 4.4)	Needed for CDC
[ ]	SQL Server Agent running (capture and cleanup jobs run under it)	Stopped Agent = no changes captured
[ ]	Log backups use native SQL Server backups, and the log is not truncated before DMS consumes it	Avoid missed changes
[ ]	Enough log disk for the full-load period	Log cannot be reused until changes are processed
[ ]	ADR (accelerated database recovery) is off for replicated databases	Not supported
[ ]	Network: DMS private IP allowed to the SQL port; TLS settings agreed	Connectivity
[ ]	Tables without primary keys identified	CDC and validation behaviour

4.2 Create the DMS login

Create a dedicated login for DMS. The exact privileges depend on whether you use MS-REPLICATION or MS-CDC and on whether the source is self-managed or RDS; the DMS documentation topic 'Permissions for SQL Server tasks' lists them. A common pattern is shown below.

```
-- On the SOURCE server
CREATE LOGIN dms_user WITH PASSWORD = '<long-random-password>', CHECK_POLICY = ON;
GRANT VIEW SERVER STATE TO dms_user;               -- read log/CDC state and metrics

USE SalesDB;
CREATE USER dms_user FOR LOGIN dms_user;
ALTER ROLE db_datareader ADD MEMBER dms_user;      -- full load needs read on all tables
-- For MS-CDC on RDS or self-managed (reads cdc.* tables and CDC metadata):
ALTER ROLE db_owner ADD MEMBER dms_user;
-- For MS-REPLICATION on a self-managed server, DMS usually needs sysadmin:
-- ALTER SERVER ROLE sysadmin ADD MEMBER dms_user;   -- remove after the migration
```

Least privilege

Use a named login that you disable or drop after the migration. Store the password in AWS Secrets Manager and reference it from the endpoint. On RDS the master user is a common shortcut in tests, but a dedicated user is better for production.

4.3 Recovery model and transaction log management

CDC and replication **depend on the transaction log**. If SQL Server truncates the log before DMS has read a change, that change is lost and you must reload. Plan for these rules:

- **FULL recovery model** for every database being replicated.
- **Log backups** must continue (the log still needs backing up in FULL), but they must be **native SQL Server backups**. Third-party backup tools that bypass native log backups can break CDC.
- The log is not marked reusable until the capture process has read it. Watch `log_reuse_wait_desc` (it shows REPLICATION while waiting) and size the log disk for the full-load duration plus a safety margin.
- Do not run `TRUNCATE_ONLY`-style tricks, switch to SIMPLE, or restore over the database during a task.
- On RDS, the platform manages log backups; use DMS version 3.5.3 or later so that DMS can also read from backed-up logs.

```
ALTER DATABASE SalesDB SET RECOVERY FULL;

-- Is the log waiting on replication/CDC? How full is it?
SELECT name, recovery_model_desc, log_reuse_wait_desc FROM sys.databases;
DBCC SQLPERF(LOGSPACE);
```

4.4 Enable change capture

Source type	Recommended mode	How
Amazon RDS for SQL Server	MS-CDC (only option)	rds_cdc_enable_db, then sp_cdc_enable_table per table
Self-managed (on-premises or EC2) with sysadmin available	MS-REPLICATION (DMS default) or MS-CDC	DMS creates the publication; you need a distributor
Self-managed, tables without primary keys, or limited privileges	MS-CDC	sp_cdc_enable_db and sp_cdc_enable_table

Option A: Amazon RDS for SQL Server source (MS-CDC)

```
-- Run as a user with sufficient rights on the RDS source
EXEC msdb.dbo.rds_cdc_enable_db 'SalesDB';

USE SalesDB;
-- Enable each table to be migrated. @supports_net_changes = 1 requires a primary key or unique index.
EXEC sys.sp_cdc_enable_table @source_schema = N'dbo', @source_name = N'Orders',
     @role_name = NULL, @supports_net_changes = 1;
EXEC sys.sp_cdc_enable_table @source_schema = N'dbo', @source_name = N'AuditLog',   -- heap, no PK
     @role_name = NULL, @supports_net_changes = 0;

-- Confirm
SELECT name, is_cdc_enabled FROM sys.databases WHERE name = 'SalesDB';
SELECT name, is_tracked_by_cdc FROM sys.tables ORDER BY name;
```

Option B: Self-managed source with MS-CDC

```
USE SalesDB;
EXEC sys.sp_cdc_enable_db;                        -- needs sysadmin; SQL Server Agent must be running
EXEC sys.sp_cdc_enable_table @source_schema = N'dbo', @source_name = N'Orders',
     @role_name = NULL, @supports_net_changes = 1;

-- Tune the capture job so latency stays low on busy systems (seconds)
EXEC sys.sp_cdc_change_job @job_type = 'capture', @pollinginterval = 5;
-- Keep change rows long enough for DMS to read them (minutes); default is 3 days
EXEC sys.sp_cdc_change_job @job_type = 'cleanup', @retention = 4320;
```

Option C: Self-managed source with MS-REPLICATION

DMS creates a publication with one article per table and relies on a **distributor**. If the server is not yet a distributor, configure distribution (SSMS: **Replication > Configure Distribution**) and make sure the distribution database is online and the DMS login can use it. Remember that the Log Reader Agent also holds the log until it has read the changes.

```
-- Is this server configured as a distributor?
EXEC sp_get_distributor;
-- Which databases are published?
SELECT name, is_published, is_cdc_enabled FROM sys.databases;
```

For **Always On availability groups** use the listener (or primary) as the source endpoint and follow the DMS topic on self-managed AlwaysOn groups. With SQL Server 2012 and 2014 the distribution database cannot be placed inside the availability group. A SQL Server **read replica** can also be used as a source in newer DMS versions.

4.5 Source endpoint settings you may need

Setting	Purpose	Guidance
SafeguardPolicy	How DMS prevents log truncation while it reads: RELY_ON_SQL_SERVER_REPLICATION_AGENT (default), EXCLUSIVE_AUTOMATIC_TRUNCATION, SHARE_AUTOMATIC_TRUNCATION	Keep the default unless you understand the trade-off. EXCLUSIVE_AUTOMATIC_TRUNCATION does not work on RDS sources.
TlogAccessMode / ReadBackupOnly	Controls whether DMS reads the active log, backups, or both (setting names depend on DMS version)	Use when log backups run frequently; check your version's documentation
UseBcpFullLoad	Use BCP for full load (faster) instead of ODBC	Often helps large loads; test
BcpPacketSize	BCP packet size	Increase for large tables if the network allows
ControlTablesFileGroup	Filegroup for the awsdms_* control tables (target use)	Useful when the target has multiple filegroups
QuerySingleAlwaysOnNode	Query only one node in an Always On group	Use if reads from a secondary cause issues
TrimSpaceInChar	Trim trailing spaces from CHAR/NCHAR	Important when validating to engines that treat trailing spaces differently
UseThirdPartyBackupDevice	Allow third-party backup devices for log access	Only if you must; native backups are preferred

4.6 Common source problems

Symptom	Cause	Fix
Log file keeps growing, disk fills	CDC/replication reader is behind or the capture job is not running	Start SQL Agent, fix the capture/reader, add log space, speed up DMS
No changes arrive	Table not enabled for CDC, wrong mode, or Agent stopped	Check is_tracked_by_cdc and Agent status
Lost events after a log backup	Log read from a truncated/archived log	Reload affected tables, use a current DMS version and backup-log access settings
Task fails with permission errors	DMS login lacks rights	Review 'Permissions for SQL Server tasks'; add VIEW SERVER STATE, db_owner or sysadmin as needed
Updates missing on a table	No primary key and MS-REPLICATION mode	Add a PK or switch to MS-CDC

5. Preparing the Target

The most common reason for a slow or failing homogeneous DMS migration is not DMS itself but a target that was not prepared. Decide how the schema gets there, in what order objects are created, and what must be switched off during the load.

5.1 Target SQL Server (RDS or EC2)

- Create the target instance with the **same collation and time zone** as the source (see the RDS guide). Use the same or a newer SQL Server version.
- Create the **database** and a **target login** for DMS with `db_owner` on that database. DMS creates its own `awsdms_*` control tables there.
- Size storage for data **plus** index space and log growth during the load. Temporarily use a larger instance class if the load is write-heavy; scale down after cutover.
- On RDS, keep the target **Single-AZ during the load** and enable Multi-AZ afterwards to avoid synchronous replication overhead and long conversions.
- Disable or move the **maintenance window** away from the migration. Keep backup retention on, but expect backup-related I/O.

5.2 Schema: what to create before, during and after

In a SQL Server to SQL Server migration DMS creates only basic tables (columns and primary keys) if you let it. For a production move, **create the schema yourself** and tell DMS to leave tables alone (`DO_NOTHING` or `TRUNCATE_BEFORE_LOAD`).

Object	When to create	Reason
Tables with data types, identity, computed columns, defaults	Before load	DMS does not recreate identity or computed behaviour
Primary keys and unique constraints	Before load	CDC apply, validation and batch apply rely on them
Nonclustered indexes	After full load (before cutover)	Faster bulk load; build while CDC continues
Foreign keys, check constraints	After full load, validated before cutover	FKs break parallel load order
Triggers	After cutover (or disabled until then)	Triggers would double-apply changes already replicated
Views, procedures, functions	Anytime before cutover	They do not affect data load
Jobs, logins, linked servers, mail	Before cutover	Not handled by DMS

Ways to get the schema onto the target

Method	How	Notes
SSMS Generate Scripts	Schema only, choose objects and options	Simple; review ordering and dependencies
SqlPackage (DACPAC)	Extract a DACPAC from the source, publish to the target	Repeatable and scriptable; includes all objects
Native backup/restore of a copy	Restore on a scratch server, drop the data, script it	Rarely needed
DMS creates tables (DROP_AND_CREATE)	DMS makes basic tables	Only for quick tests; you must add everything else afterwards

```
# Extract schema only from the source and publish it to the target (database must already exist)
sqlpackage /Action:Extract /SourceConnectionString:"Server=src01;Database=SalesDB;User
    Id=dms_admin;Password=***;TrustServerCertificate=True" ^
  /TargetFile:SalesDB.dacpac /p:ExtractAllTableData=False
sqlpackage /Action:Publish /SourceFile:SalesDB.dacpac ^
  /TargetConnectionString:"Server=mydb.xxxx.ap-south-1.rds.amazonaws.com,1433;Database=SalesDB;User
      Id=admin;Password=***;Encrypt=True"
```

If you publish the full schema (with FKs, indexes and triggers) before the load, switch the troublesome objects off for the duration of the load and switch them on afterwards:

```
-- Generate commands to DISABLE foreign keys and triggers before the full load
SELECT 'ALTER TABLE ' + QUOTENAME(s.name) + '.' + QUOTENAME(t.name) + ' NOCHECK CONSTRAINT ALL;'
FROM sys.tables t JOIN sys.schemas s ON s.schema_id = t.schema_id;
SELECT 'ALTER TABLE ' + QUOTENAME(s.name) + '.' + QUOTENAME(t.name) + ' DISABLE TRIGGER ALL;'
FROM sys.tables t JOIN sys.schemas s ON s.schema_id = t.schema_id
WHERE EXISTS (SELECT 1 FROM sys.triggers tr WHERE tr.parent_id = t.object_id);

-- After the full load: re-enable and VALIDATE constraints (WITH CHECK makes them trusted)
SELECT 'ALTER TABLE ' + QUOTENAME(s.name) + '.' + QUOTENAME(t.name) + ' WITH CHECK CHECK CONSTRAINT
    ALL;'
FROM sys.tables t JOIN sys.schemas s ON s.schema_id = t.schema_id;
```

5.3 Identity columns and sequences

- The identity **property is not migrated** from the source, so create the target column with `IDENTITY` yourself. DMS loads the explicit values; test this on one table first.
- After the load and just before cutover, **reseed** each identity so new rows continue after the highest migrated value.
- For sequence objects, create them on the target and set their current value from the source at cutover.

```
-- At cutover: set identity to the current max (run per table)
DBCC CHECKIDENT ('dbo.Orders', RESEED);          -- reseeds to the current maximum value
-- Sequences: restart at the source's next value
ALTER SEQUENCE dbo.InvoiceSeq RESTART WITH 1500001;
```

5.4 Other target types in brief

Target	Preparation	Where to read more
Aurora / RDS PostgreSQL or MySQL	Convert schema and code with DMS Schema Conversion; create tables without FKs/indexes; map data types	Chapter 9
Amazon S3 (data lake)	Create bucket and IAM role; choose Parquet or CSV; plan folder layout and CDC file semantics	DMS S3 target documentation
Amazon Redshift	Create cluster, staging S3 bucket and role; distribution/sort keys	DMS Redshift target documentation

5.5 Target readiness checklist

Done?	Item
[ ]	Target database created with correct collation, compatibility level and file layout
[ ]	DMS login with db_owner; credentials stored in Secrets Manager
[ ]	Tables, PKs, identity and computed columns created; secondary objects scripted for later
[ ]	FKs and triggers disabled (or not yet created) during the load
[ ]	Storage, IOPS and instance class sufficient for the load; Single-AZ for the load window
[ ]	Security group allows 1433 from the DMS security group only; SSL configured
[ ]	Monitoring (CloudWatch, Performance Insights) enabled on the target

6. Building the DMS Migration, Step by Step

This chapter creates every DMS object, using the console concepts and the AWS CLI. Names and addresses are examples; replace them with your own.

6.1 Step 1: IAM roles and prerequisites

- **dms-vpc-role** (trusted by dms.amazonaws.com, AWS managed policy `AmazonDMSVPCManagementRole`) - required before you can create a replication instance with the CLI or API.
- **dms-cloudwatch-logs-role** (managed policy `AmazonDMSCloudWatchLogsRole`) - lets DMS write task logs.
- A role for **Secrets Manager access** if endpoints use secrets, and an S3 role/bucket for premigration assessment results.
- Your own IAM users/roles need permissions for `dms:*` actions, plus EC2/VPC and KMS as required.

6.2 Step 2: Subnet group and replication instance

```
aws dms create-replication-subnet-group --replication-subnet-group-identifier dms-subnets \
  --replication-subnet-group-description "DMS private subnets" \
  --subnet-ids subnet-0aaa1111 subnet-0bbb2222

aws dms create-replication-instance --replication-instance-identifier dms-sql-01 \
  --replication-instance-class dms.r5.xlarge --allocated-storage 200 --multi-az \
  --vpc-security-group-ids sg-0dms0001 --replication-subnet-group-identifier dms-subnets \
  --no-publicly-accessible --kms-key-id alias/dms-key
```

Wait until the instance status is **available**. Then confirm from the instance's private IP that you can reach both databases (allow it in firewalls and security groups).

6.3 Step 3: Endpoints

```
# Source: SQL Server (on-premises or EC2). Credentials come from Secrets Manager.
aws dms create-endpoint --endpoint-identifier src-sql-onprem --endpoint-type source \
  --engine-name sqlserver --server-name sql01.corp.local --port 1433 --database-name SalesDB \
  --ssl-mode require \
  --microsoft-sql-server-settings
      '{"SecretsManagerAccessRoleArn":"arn:aws:iam::111122223333:role/dms-secrets-role",
                                  "SecretsManagerSecretId":"dms/sql-source"}'

# Target: Amazon RDS for SQL Server
aws dms create-endpoint --endpoint-identifier tgt-rds-sql --endpoint-type target \
  --engine-name sqlserver --server-name mydb.xxxx.ap-south-1.rds.amazonaws.com --port 1433 \
  --database-name SalesDB --username dms_target --password '<password>' --ssl-mode require

# Test each connection from the replication instance
aws dms test-connection --replication-instance-arn <instance-arn> --endpoint-arn <source-arn>
aws dms describe-connections --filters Name=endpoint-arn,Values=<source-arn>
```

A named instance uses the `host\\instance` notation in the server name together with the real port. After **test-connection** returns **successful** for both endpoints, you can create a task.

6.4 Step 4: Table mappings

Table mappings are JSON **rules**. **Selection** rules include or exclude tables, **transformation** rules rename or reshape, and **table-settings** rules control parallel load and filters.

```
{
  "rules": [
    { "rule-type": "selection", "rule-id": "1", "rule-name": "include-dbo",
      "object-locator": { "schema-name": "dbo", "table-name": "%" }, "rule-action": "include" },
    { "rule-type": "selection", "rule-id": "2", "rule-name": "skip-staging",
      "object-locator": { "schema-name": "dbo", "table-name": "stg_%" }, "rule-action": "exclude" },
    { "rule-type": "selection", "rule-id": "3", "rule-name": "recent-orders-only",
      "object-locator": { "schema-name": "dbo", "table-name": "OrderArchive" }, "rule-action":
          "include",
      "filters": [ { "filter-type": "source", "column-name": "OrderDate",
                     "filter-conditions": [ { "filter-operator": "gte", "value": "2023-01-01" } ] } ] },
    { "rule-type": "table-settings", "rule-id": "4", "rule-name": "parallel-big-table",
      "object-locator": { "schema-name": "dbo", "table-name": "Orders" },
      "parallel-load": { "type": "ranges", "columns": ["OrderID"],
                         "boundaries": [["10000000"], ["20000000"], ["30000000"]] } }
  ]
}
```

Use **ranges** (or partitions, where supported) to load a very large table with several threads in parallel. Choose boundaries that split the data evenly and are on an indexed column.

6.5 Step 5: Task settings

Setting	What it does	Recommendation for SQL Server
TargetTablePrepMode	DROP_AND_CREATE, TRUNCATE_BEFORE_LOAD or DO_NOTHING	DO_NOTHING (empty pre-created tables) or TRUNCATE_BEFORE_LOAD for reruns
MaxFullLoadSubTasks	Number of tables loaded in parallel (default 8, max 49)	Raise gradually while watching CPU, memory and the source
LOB handling: SupportLobs, LimitedSizeLobMode, LobMaxSize, FullLobMode, InlineLobMaxSize	How large objects are transferred	Limited LOB mode with the true maximum size is fastest; use full LOB only when needed
BatchApplyEnabled (change processing)	Apply CDC changes in batches	Helps high-volume CDC; works best with primary keys; test
CommitRate / MinTransactionSize	Rows per commit in full load / change apply	Increase for throughput if the target log allows
EnableValidation	Compare source and target data during the migration	On for production; adds load, so size accordingly
StopTaskCachedChangesApplied / NotApplied	Pause automatically at key moments	Use to create indexes before CDC starts
CreatePkAfterFullLoad	Defers creating PKs until load ends (when DMS creates tables)	Not needed with pre-created tables
Logging (CloudWatch)	Task logs by component	Enable; raise to detail only when debugging
ErrorBehavior / TableErrorPolicy	What happens when a table or record fails	SUSPEND_TABLE keeps other tables running

```
{
  "TargetMetadata": { "SupportLobs": true, "LimitedSizeLobMode": true, "LobMaxSize": 32,
      "BatchApplyEnabled": true },
  "FullLoadSettings": { "TargetTablePrepMode": "DO_NOTHING", "MaxFullLoadSubTasks": 8, "CommitRate":
      10000,
                        "StopTaskCachedChangesApplied": false },
  "ValidationSettings": { "EnableValidation": true, "ThreadCount": 5 },
  "ChangeProcessingTuning": { "BatchApplyTimeoutMin": 1, "BatchApplyTimeoutMax": 30,
      "MinTransactionSize": 1000,
                             "MemoryLimitTotal": 1024, "MemoryKeepTime": 60 },
  "Logging": { "EnableLogging": true }
}
```

This is an excerpt for orientation. Start from the console's default settings and change only what you have tested. LobMaxSize is in KB.

6.6 Step 6: Premigration assessment

Before the first start, run a **premigration assessment**. It inspects the task, endpoints and source for known problems (unsupported data types, missing primary keys, LOB sizes, permissions, configuration) and writes a report to S3. Fix everything it reports as an error.

```
aws dms start-replication-task-assessment-run --replication-task-arn <task-arn> \
  --service-access-role-arn arn:aws:iam::111122223333:role/dms-assessment-role \
  --result-location-bucket my-dms-assessments --assessment-run-name sales-pre-run-1
```

6.7 Step 7: Create and start the task

```
aws dms create-replication-task --replication-task-identifier sales-full-cdc \
  --source-endpoint-arn <source-arn> --target-endpoint-arn <target-arn> \
  --replication-instance-arn <instance-arn> --migration-type full-load-and-cdc \
  --table-mappings file://table-mappings.json --replication-task-settings file://task-settings.json

aws dms start-replication-task --replication-task-arn <task-arn> --start-replication-task-type
    start-replication
aws dms describe-replication-tasks --filters Name=replication-task-arn,Values=<task-arn>
```

Start type	Use
start-replication	First start of a task
resume-processing	Continue from where a stopped task left off (CDC position kept)
reload-target	Reload all tables and start again (full load again)

For a **CDC-only** task (for example after seeding the target with a native backup), start from a known point with `--cdc-start-time` or a log position (`--cdc-start-position`). Choose a point **earlier** than the backup you restored so no change is missed; DMS applies idempotently where it can, but test the behaviour for your tables.

6.8 Alternative: DMS Serverless

```
aws dms create-replication-config --replication-config-identifier sales-serverless \
  --source-endpoint-arn <source-arn> --target-endpoint-arn <target-arn> \
  --replication-type full-load-and-cdc --table-mappings file://table-mappings.json \
  --compute-config
      "MinCapacityUnits=2,MaxCapacityUnits=32,MultiAZ=false,ReplicationSubnetGroupId=dms-subnets,VpcSecu
      rityGroupIds=sg-0dms0001"

aws dms start-replication --replication-config-arn <config-arn> --start-replication-type
    start-replication
```

One **DCU** is about 2 GB of memory. Set a maximum high enough for the busiest hour and a minimum low enough to save cost. DMS provisions and scales capacity for you and reports it in the replication status.

7. Running the Migration: Full Load, CDC and Validation

Once the task starts, your job changes from building to watching: is the load moving, is latency under control, and does the target data match the source?

7.1 What to expect

- **Task status** moves from *Starting* to *Running*. Tables show *Before load*, *Table loading* and then *Table completed*.
- **Full load** runs several tables in parallel (the default is eight) and the biggest tables decide the finish time.
- **Cached changes** made on the source during the load are applied after it completes.
- **CDC** starts, and the latency metrics fall toward a few seconds. Your task is now ready for cutover whenever you are.

7.2 Monitoring

Watch three places: the DMS console (task overview, **table statistics**, monitoring graphs, logs), CloudWatch alarms, and the **control tables** on the target.

CloudWatch metric (AWS/DMS)	Meaning	Watch for
CDCLatencySource	Seconds between the last change on the source and the last change DMS captured	Rising: DMS cannot read fast enough or the source reader is behind
CDCLatencyTarget	Seconds between the last change on the source and its application on the target	Rising: target writes are slow or the instance is under-sized
CDCIncomingChanges	Changes waiting to be applied	A growing backlog
FullLoadThroughputRowsTarget	Rows/second written during the load	Compare against your test baseline
CPUUtilization / FreeableMemory	Replication instance health	CPU high or memory low means a bigger class is needed
SwapUsage / FreeStorageSpace	Memory spill and instance disk	Swap use or low disk means changes are spilling to disk: add memory/storage
NetworkReceive/TransmitThroughput	Data volume on the instance	Near the instance network limit

```
# Per-table progress (state, rows loaded, changes applied, validation)
aws dms describe-table-statistics --replication-task-arn <task-arn> \
  --query "TableStatistics[].{schema:SchemaName,tbl:TableName,state:TableState,full:FullLoadRows,ins:Ins
      erts,upd:Updates,del:Deletes,val:ValidationState}" \
  --output table

# Alarm on target latency above 60 seconds for 5 minutes
aws cloudwatch put-metric-alarm --alarm-name dms-sales-latency --namespace AWS/DMS \
  --metric-name CDCLatencyTarget --statistic Average --period 60 --evaluation-periods 5 --threshold 60 \
  --comparison-operator GreaterThanThreshold \
  --dimensions Name=ReplicationInstanceIdentifier,Value=dms-sql-01
      Name=ReplicationTaskIdentifier,Value=<task-resource-id> \
  --alarm-actions arn:aws:sns:ap-south-1:111122223333:dba-alerts
```

```
-- On the TARGET: DMS control tables
SELECT TOP 50 * FROM dbo.awsdms_apply_exceptions ORDER BY ERROR_TIME DESC;   -- rows DMS could not
    apply, with the error
SELECT * FROM dbo.awsdms_status;                                            -- task, pending changes,
    positions
SELECT * FROM dbo.awsdms_suspended_tables;                                   -- tables suspended by
    errors
```

7.3 Table and task states

State	Meaning	Action
Before load / Table loading	Waiting or loading	Normal
Table completed	Full load done and CDC active for that table	Normal
Table error	A table failed (data error, constraint, permission)	Read the exception table and logs; fix; reload the table
Table cancelled / suspended	Stopped because of a policy or error	Fix cause; reload
Task: Running / Stopped / Failed / Ready	Overall task state	Failed: read task logs first

7.4 Data validation

With **EnableValidation** on, DMS compares rows on the source and target after they are loaded and keeps comparing as changes are applied. It reports tables as validated, pending, mismatched or suspended. Validation requires a **primary key or unique index** on each table and adds read load on both databases.

Validation state	Meaning
Validated	Source and target rows match
Pending / not enough data	Still comparing or waiting for changes to settle
Mismatched records	Rows differ: investigate data types, trailing spaces, time precision, collation
Suspended records	Validation gave up on rows after repeated failures
No primary key	Table cannot be validated

- Typical false mismatches: **trailing spaces** in CHAR columns, different **datetime precision**, floating-point rounding and **collation** differences. Set `TrimSpaceInChar` and compare types before blaming DMS.
- Validation does not replace business checks. Also compare **row counts, checksums and key reports** yourself before cutover.
- Newer DMS versions (3.6.1) add **data resync** for SQL Server sources with PostgreSQL targets, which can automatically fix mismatches found by validation.

7.5 Performance tuning

Symptom	Likely bottleneck	What to try
Slow full load, source busy	Source read speed or locks	Load off-peak; use a read replica or restored copy as source for the initial load; avoid heavy source jobs
Slow full load, target busy	Target I/O, log, indexes or triggers	Scale target IOPS/instance; disable indexes, FKs and triggers; use TRUNCATE_BEFORE_LOAD on empty tables
One huge table dominates	Single-threaded load	Parallel-load ranges/partitions; raise MaxFullLoadSubTasks if CPU allows
Instance CPU 100%	Under-sized or transformation-heavy	Larger or c-class instance; simplify transformations
Instance swapping / low memory	Big transactions or LOBs	r-class instance; limited LOB mode; reduce LobMaxSize; increase memory limits
High CDC latency	Apply rate or capture rate too low	Enable batch apply; tune MinTransactionSize/CommitRate; speed capture polling; bigger instance
Latency spikes at batch jobs	Large source transactions	Throttle the batch, schedule away from cutover, increase memory
Network saturated	Link or VPN tunnel	Direct Connect; more tunnels; compression on the path; move the instance closer

7.6 Managing a running task

```
# Pause and resume (CDC position is kept)
aws dms stop-replication-task --replication-task-arn <task-arn>
aws dms start-replication-task --replication-task-arn <task-arn> --start-replication-task-type
    resume-processing

# Reload one table (for example after fixing a data error)
aws dms reload-tables --replication-task-arn <task-arn> --tables-to-reload
    SchemaName=dbo,TableName=Orders

# Change table mappings or settings (task must be stopped), then resume
aws dms modify-replication-task --replication-task-arn <task-arn> --table-mappings
    file://table-mappings-v2.json
```

- **Adding tables:** stop the task, update the mappings, enable CDC for the new tables on the source, resume, and reload the new tables.
- **Schema changes (DDL):** support depends on mode and DMS version and is limited. Safest practice: apply the DDL to the target first, then to the source, and test it in the rehearsal. Avoid schema changes during a migration where possible.
- **Source maintenance:** do not restore over, detach, or switch recovery models on a replicated database while the task runs.
- **While CDC runs** you can create nonclustered indexes on the target, refresh statistics and run read tests of the application against the target.

8. Cutover, Rollback and Clean-up

Cutover is the short, planned moment when the application stops writing to the old database and starts writing to the new one. With DMS the data is already there; the work is finishing the schema, proving the data and switching.

8.1 Readiness criteria

Criterion	Target
Latency (source and target)	Near zero for a sustained period, including during a peak
Task status	Running, no table errors or suspended tables
Validation	All tables validated, or mismatches explained
Secondary objects	Indexes created; FK/check scripts tested; triggers ready; statistics updated
Server-level objects	Logins, jobs, linked servers, mail re-created and tested
Performance	Application tested against the target with production-like load
Rollback plan	Agreed, with a decision time
Rehearsal	At least one full dress rehearsal with timings

8.2 Cutover sequence

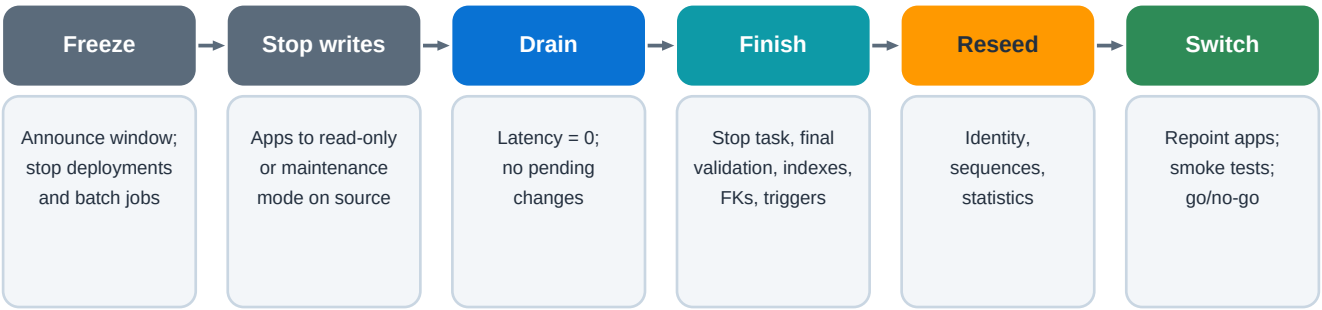


Figure 8.1 - Cutover sequence for a DMS migration

When	Action	Owner
T - 1 week	Dress rehearsal complete; go/no-go review; schedule communicated	Project lead
T - 1 day	Confirm latency stable, validation clean, scripts tested; lower DNS TTL	DBA
T - 30 min	Stop deployments, batch jobs and schedulers on the source	DBA / App
T - 15 min	Put application in maintenance mode or read-only; stop writes	App team
T	Wait for CDCLatency = 0 and no pending changes; stop the task: `aws dms stop-replication-task`	DBA
T + 5 min	Final row counts and checksums; finish validation	DBA
T + 10 min	Apply remaining objects: FKs (WITH CHECK), triggers, secondary indexes if not done; reseed identities and sequences; update statistics	DBA
T + 25 min	Repoint application connection strings or DNS; start application; smoke tests	App team
T + 2 hours	Go/no-go. If no-go: roll back (section 8.4)	Project lead

```
-- Typical final steps on the SQL Server target
EXEC sp_MSforeachtable 'ALTER TABLE ? WITH CHECK CHECK CONSTRAINT ALL';     -- validate FKs/checks (or
    use generated scripts)
EXEC sp_MSforeachtable 'ALTER TABLE ? ENABLE TRIGGER ALL';
EXEC sp_updatestats;
DBCC CHECKIDENT ('dbo.Orders', RESEED);                                     -- per identity table
```

8.3 Objects DMS did not move

Object	How to move it
Logins and permissions	Script logins with SIDs and hashes; recreate server roles; fix orphaned users (ALTER USER ... WITH LOGIN)
SQL Agent jobs, operators, alerts	Script from SSMS; adjust for RDS limitations; replace CmdExec/PowerShell steps
Linked servers, Database Mail, credentials	Recreate and test from the new network location
Views, procedures, functions, user-defined types	Included in the schema scripts you applied; verify counts match
SSIS/SSRS/SSAS content	Re-host or redesign; not part of a table data migration

8.4 Rollback options

Option	How	Trade-off
Fall back to the source	Keep the source intact and unchanged; point applications back	Writes made on the target after cutover are lost unless copied back
Reverse replication	Prepare a second DMS task from the new database back to the old one (enable CDC on the new source, schema in place)	Extra build effort; keeps both sides in sync during the rollback window
Restore from backup	Point-in-time restore of the target or a pre-cutover snapshot	Slower; for corruption, not for a quick switch back

> **Decide the rollback point in advance**
> State a clock time after which rollback is no longer simple (for example once users have created business data on the new system). Agree who can call it and what rolling back means for in-flight transactions.

8.5 Clean-up

```
# 1) Stop and delete DMS resources
aws dms stop-replication-task --replication-task-arn <task-arn>
aws dms delete-replication-task --replication-task-arn <task-arn>
aws dms delete-endpoint --endpoint-arn <source-arn>
aws dms delete-endpoint --endpoint-arn <target-arn>
aws dms delete-replication-instance --replication-instance-arn <instance-arn>
```

```
-- 2) On the SOURCE: remove change capture / replication artifacts (they are NOT removed when the task
    is deleted)
USE SalesDB;
EXEC sys.sp_cdc_disable_db;                -- if you used MS-CDC
EXEC sp_removedbreplication N'SalesDB';    -- if DMS created a publication (MS-REPLICATION)
-- Then drop the DMS login, and remove the distributor if it was set up only for this migration

-- 3) On the TARGET: drop DMS control tables when no longer needed
DROP TABLE IF EXISTS dbo.awsdms_apply_exceptions, dbo.awsdms_status, dbo.awsdms_suspended_tables,
    dbo.awsdms_history;
```

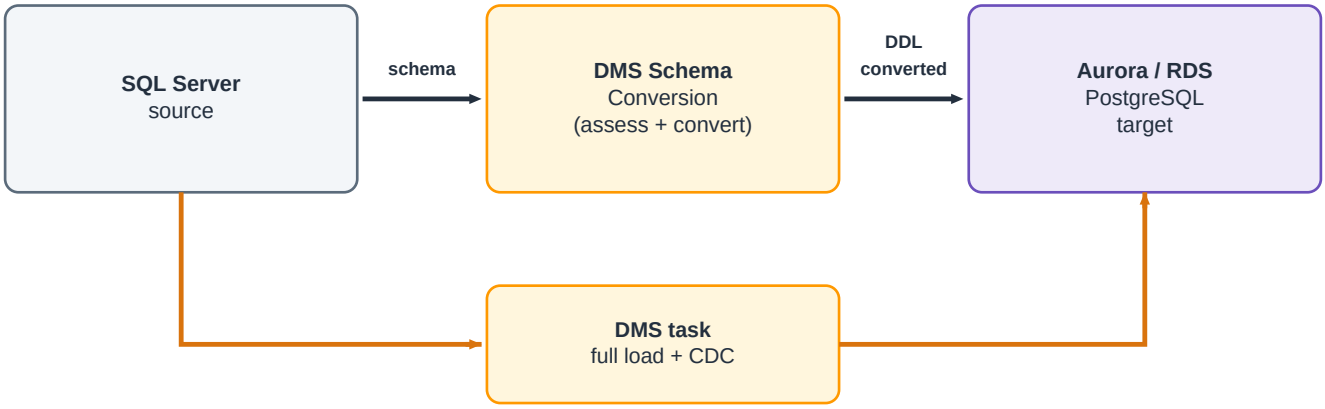
- Re-enable **Multi-AZ**, set backup retention and the maintenance window, and take a baseline snapshot on RDS.
- Delete or archive **Secrets Manager** secrets, S3 assessment reports and temporary security-group rules.
- Keep the source in **read-only** state for the agreed period, then decommission it.
- Record timings, issues and settings in a **lessons-learned** note for the next wave.

8.6 After cutover: stabilise

Task	Detail
Integrity	Run DBCC CHECKDB on the target; compare record counts and key reports again
Performance	Compare Query Store/wait stats with the source baseline; tune indexes and parameters; right-size the instance
Statistics and compatibility	Update statistics; keep the compatibility level until tests pass, then raise it deliberately
Operations	Backups verified, monitoring and alarms active, maintenance jobs scheduled, runbooks updated
Security	Remove migration access, rotate credentials, confirm TLS and least privilege

9. Heterogeneous Migration: SQL Server to PostgreSQL

Moving from SQL Server to Aurora or RDS for PostgreSQL changes the engine, so three things must be migrated: the **schema**, the **data** and the **application code**. DMS Schema Conversion handles the first, DMS handles the second, and your team handles the third.



Data path (orange): rows are read from SQL Server and written to the converted tables. Application code must be changed separately.

Figure 9.1 - Schema, data and application paths in a heterogeneous migration

9.1 Choosing the approach

Approach	When	Notes
DMS Schema Conversion + DMS data migration to PostgreSQL	Most OLTP databases; you accept application changes	Schema conversion is a free DMS feature (you pay for S3 storage). Needs manual work on complex code.
AWS Schema Conversion Tool (desktop)	Data warehouse schemas, big data frameworks, application SQL code conversion	Use where DMS Schema Conversion does not cover the case
Aurora PostgreSQL with Babelfish	Applications that must keep T-SQL and TDS drivers with minimal change	DMS supports Babelfish as a target; test coverage carefully
Stay on SQL Server (RDS or EC2)	Heavy T-SQL estate, short timeline	Homogeneous migration (chapters 4-8)

9.2 Schema conversion with DMS Schema Conversion

Step	Action
1	Create an instance profile and add **data providers** for the source SQL Server and the target PostgreSQL (credentials in Secrets Manager; network access from the DMS subnet group; an S3 bucket for reports and generated SQL).
2	Create a **migration project** and generate the **assessment report**: how many objects convert automatically and which need manual action items.
3	Select schemas and **convert**. Review the converted tables, indexes, views, procedures and functions in the console.
4	Fix action items (rewrite unsupported constructs). Save the converted SQL.
5	**Apply** the converted schema to the target (tables first). Create foreign keys, secondary indexes and triggers after the data load.
6	Repeat as the source changes during the project; keep the converted DDL in version control.

9.3 Typical code conversions

SQL Server (T-SQL)	PostgreSQL (PL/pgSQL / SQL)	Comment
TOP (n)	LIMIT n	Rewrite queries
ISNULL(a, b)	COALESCE(a, b)	Standard SQL
GETDATE(), SYSDATETIME()	now(), clock_timestamp()	Check time zone behaviour
a + b (string concatenation)	a || b or concat()	NULL handling differs
IDENTITY(1,1)	GENERATED ... AS IDENTITY or serial	Reseed with setval after load
#temp tables, table variables	TEMP tables	Scope rules differ
TRY...CATCH, @@ERROR, RAISERROR	EXCEPTION blocks, RAISE	Error model differs
MERGE	INSERT ... ON CONFLICT or MERGE (newer versions)	Test semantics
Cursors, dynamic SQL (EXEC)	Cursors, EXECUTE	Syntax changes
@@ROWCOUNT	GET DIAGNOSTICS ... ROW_COUNT	Procedure rewrite
Case-insensitive default collation	Case-sensitive by default	Use CITEXT or collation settings; test comparisons

9.4 Data type mapping (typical)

SQL Server	PostgreSQL	Notes
int, bigint, smallint	integer, bigint, smallint	
bit	boolean	Application changes from 0/1
decimal, numeric, money	numeric	money mapped to numeric(19,4)
float, real	double precision, real	
char, varchar, nvarchar(n)	char, varchar	PostgreSQL uses UTF-8 for text
varchar(max), nvarchar(max), text	text	LOB handling in the task
datetime, datetime2, smalldatetime	timestamp	Precision differences can affect validation
datetimeoffset	timestamptz	
uniqueidentifier	uuid	Binary(16) to uuid supported in newer DMS versions
varbinary, image	bytea	LOB
xml	xml or text	
geography, geometry	PostGIS types	Use limited or inline LOB mode in DMS

Actual conversions are shown in the DMS Schema Conversion output. Verify the mapping for your columns and indexes.

9.5 Loading data into PostgreSQL

- PostgreSQL folds unquoted names to **lower case**. Add transformation rules so schemas, tables and columns are created in lower case, as the converted DDL expects.
- Create **tables without foreign keys and secondary indexes** before the load, and add them afterwards. Triggers and FK checks are bypassed during the load by DMS on PostgreSQL targets.
- Use **limited LOB mode** with correct maximum sizes; validate geography/geometry columns carefully.
- After the load, **reset sequences** with `setval` to the maximum key values.
- Run **validation**, and then application regression tests; performance tuning (indexes, query plans, vacuum/analyze) is part of the migration.

```
{ "rule-type": "transformation", "rule-id": "10", "rule-name": "lower-schema", "rule-target": "schema",
  "object-locator": { "schema-name": "%" }, "rule-action": "convert-lowercase" },
{ "rule-type": "transformation", "rule-id": "11", "rule-name": "lower-table", "rule-target": "table",
  "object-locator": { "schema-name": "%", "table-name": "%" }, "rule-action": "convert-lowercase" },
{ "rule-type": "transformation", "rule-id": "12", "rule-name": "lower-column", "rule-target": "column",
  "object-locator": { "schema-name": "%", "table-name": "%", "column-name": "%" }, "rule-action":
      "convert-lowercase" }
```

9.6 Heterogeneous migration checklist

Done?	Item
[ ]	Assessment report reviewed; effort for manual conversion estimated
[ ]	Schema converted, applied and under version control
[ ]	Application code, drivers, ORM dialect and connection strings updated and tested
[ ]	Data type, collation and case-sensitivity differences tested
[ ]	DMS task loads and replicates; validation clean; sequences reset
[ ]	Performance tested with production-like queries; indexes and parameters tuned
[ ]	Cutover and rollback plan rehearsed

10. Troubleshooting and Best Practices

Almost every DMS problem shows up in one of three places: the **task log**, the **table statistics** or the **control tables**. Start there, then use this table.

10.1 Troubleshooting guide

Symptom	Likely cause	What to do
Test connection fails	Security group, route, firewall, DNS, wrong port or named instance, SSL mismatch, wrong credentials	Check the instance's private IP can reach the host and port; verify SSL mode and certificate; confirm SQL authentication (Windows auth is unsupported)
Task fails to start	Missing IAM role (dms-vpc-role, logs role), subnet group issue, invalid mappings JSON	Read the error; fix roles and JSON; rerun the premigration assessment
No changes replicated	CDC not enabled on the table, SQL Agent stopped, task is full load only	Check is_tracked_by_cdc, Agent state and task type
Log file on the source keeps growing	Reader/capture behind or stopped	Start the capture/log reader; speed DMS; add space; remember log reuse waits on REPLICATION
High CDC latency	Under-sized instance, large transactions, slow target, no batch apply	See section 7.5
Lost changes after a long outage	Log truncated/archived before DMS read it	Reload affected tables or restart with a full load
Duplicate key or constraint errors on apply	FKs/triggers/unique indexes on the target, or rows already present	Disable FKs/triggers; use TRUNCATE_BEFORE_LOAD; check apply exceptions table
'No record found for applying an UPDATE'	Target missing the row (for example failed table load)	Reload the table or use the error-handling option to insert missing rows
Updates ignored on some tables	No primary key with MS-REPLICATION	Add PK or use MS-CDC
LOB data truncated	LobMaxSize too small in limited LOB mode	Raise LobMaxSize or use full LOB for those tables
Validation mismatches	Trailing spaces, datetime precision, rounding, collation	Set TrimSpaceInChar; align types; compare samples manually
Instance out of memory or swapping	Many tasks, big transactions, LOBs	r-class instance; fewer parallel tasks; limited LOB mode
Replication instance storage full	Log and swap files	Allocate more storage; reduce logging detail
Identity values start low after cutover	Identity property not migrated or not reseeded	DBCC CHECKIDENT RESEED, sequence restart
Time zone or date shifts	Mismatched time zone/type mapping	Align types and server time zone; check datetime precision
Task works in test but slow in production	More data, busier source, network differences	Re-test with production-sized data; adjust parallelism and instance

10.2 Reading task logs

- Turn on task logging to CloudWatch. Components include **SOURCE_CAPTURE**, **SOURCE_UNLOAD**, **TARGET_LOAD**, **TARGET_APPLY** and **TASK_MANAGER**.
- Use the default severity for normal runs. Raise a specific component to **detailed debug** only while diagnosing, then lower it (debug logs are large).
- Search CloudWatch Logs for `E:` (errors) and `W:` (warnings) first.
- Open an AWS support case with the task ARN, DMS version and relevant log excerpts if the cause is not clear.

10.3 Best practices

Area	Practice
Planning	Rehearse the whole migration; run the premigration assessment; keep migration scripts in version control
Schema	Create schema yourself for homogeneous moves; load data without FKs/triggers; build secondary indexes after the load
Source	FULL recovery, native log backups, enough log space, SQL Agent running, tested permissions
Sizing	Memory-optimised instance for CDC; test throughput early; use parallel load for large tables
Security	Private instance, SSL on endpoints, Secrets Manager, KMS, least-privilege logins, remove access afterwards
Monitoring	Alarms on latency, CPU, memory, storage and task state; review table statistics daily
Cutover	Written runbook with times, owners, rollback decision; confirm zero latency before stopping writes
Cost	Stop or delete idle tasks and instances; right-size; consider Serverless for variable load; clean up CDC objects
Documentation	Record settings, issues and timings for the next wave

Appendix A: DMS CLI Cheat Sheet

Task	Command
Create subnet group	aws dms create-replication-subnet-group --replication-subnet-group-identifier ... --subnet-ids ...
Create replication instance	aws dms create-replication-instance --replication-instance-identifier ... --replication-instance-class dms.r5.xlarge ...
Create endpoint	aws dms create-endpoint --endpoint-type source|target --engine-name sqlserver ...
Test connection	aws dms test-connection --replication-instance-arn ... --endpoint-arn ...
Create task	aws dms create-replication-task --migration-type full-load-and-cdc --table-mappings file://... --replication-task-settings file://...
Premigration assessment	aws dms start-replication-task-assessment-run --replication-task-arn ... --result-location-bucket ...
Start / resume / reload	aws dms start-replication-task --start-replication-task-type start-replication | resume-processing | reload-target
Stop task	aws dms stop-replication-task --replication-task-arn ...
Reload a table	aws dms reload-tables --replication-task-arn ... --tables-to-reload SchemaName=dbo,TableName=Orders
Table statistics	aws dms describe-table-statistics --replication-task-arn ...
Task status	aws dms describe-replication-tasks --filters Name=replication-task-arn,Values=...
Serverless config	aws dms create-replication-config ... --compute-config MinCapacityUnits=..,MaxCapacityUnits=..
Start serverless	aws dms start-replication --replication-config-arn ... --start-replication-type start-replication
Delete resources	aws dms delete-replication-task | delete-endpoint | delete-replication-instance

Confirm option names with `aws dms help` *for your CLI version.*

Appendix B: Pre-Cutover Checklist

Done?	Item
[ ]	Task Running; CDCLatencySource/Target near zero for 24 hours; no table errors
[ ]	Validation complete; remaining mismatches explained and accepted
[ ]	Secondary indexes created and statistics updated on the target
[ ]	FK/check/trigger scripts tested in rehearsal; reseed scripts ready
[ ]	Logins, jobs, linked servers, mail re-created and tested
[ ]	Application tested against the target; connection strings and DNS changes ready
[ ]	Rollback plan, decision time and owners agreed
[ ]	Monitoring, alarms and support rota ready; stakeholders informed; change approved

Appendix C: Glossary

Term	Meaning
AWS DMS	AWS Database Migration Service
Replication instance	EC2-based compute managed by DMS that runs tasks
DMS Serverless / DCU	Automatically scaled DMS capacity; one DMS Capacity Unit is about 2 GB of memory
Endpoint	Connection definition for a source or target
Task / migration type	A unit of work: full load, CDC, or full load + CDC
Full load	One-time copy of existing data
CDC	Change data capture: ongoing replication of changes
MS-REPLICATION / MS-CDC	The two SQL Server mechanisms DMS uses to read changes
Table mappings / task settings	JSON that selects/transforms tables and tunes behaviour
Control tables	awsdms_* tables DMS creates on the target
Premigration assessment	Pre-run check of a task for known problems
DMS Schema Conversion / SCT	Tools to convert schema and code for heterogeneous migrations
Homogeneous / heterogeneous	Same engine / different engine migration
Cutover	Moment the application switches to the new database
RPO / RTO	Recovery point / time objective

Appendix D: Where to Learn More

AWS DMS changes frequently (new engine versions, features and retirements such as DMS Fleet Advisor). Treat this guide as a process reference and confirm details in the current documentation.

Topic	Where to look
DMS user guide	AWS Database Migration Service User Guide: sources, targets, tasks, table mappings, task settings, monitoring
SQL Server as a source	'Using a Microsoft SQL Server database as a source for AWS DMS' (limitations, permissions, CDC prerequisites, endpoint settings)
SQL Server as a target	'Using a Microsoft SQL Server database as a target for AWS DMS'
Release notes	AWS DMS release notes (engine versions and fixes)
Schema conversion	AWS DMS Schema Conversion and AWS Schema Conversion Tool documentation
Assessment	AWS Migration Evaluator; DMS premigration assessments
Best practices	AWS DMS best practices; AWS Prescriptive Guidance for SQL Server migrations
Pricing	AWS DMS pricing page (replication instances, serverless DCUs, storage, data transfer)